Instant Service Booking System

Team name: - CodeCrew

Team leader name: - Aman Sharma

Project Documentation

1. Problem Statement

In modern urban life, people need instant house help services such as cleaning, plumbing, electricians, cooking, and maintenance. The current booking process is time-consuming and involves multiple calls without real-time availability and estimated times of arrival.

There is a need for a fast and automated system that will allow users to instantly book nearby service providers and receive assistance within minutes.

The objective is to create a smart digital platform that will make service booking easier, faster, and more convenient through real-time automation using Firebase.

2. Objective

To build a smart and instant house help booking system that:

- **Allows users to register and log in**
- **Enables quick selection of service categories**
- **Provides instant booking functionality**
- **Generates unique booking IDs**
- **Shows estimated arrival time**
- **Maintains booking history**
- **Ensures services are delivered within 15 minutes using location-based automation**

3. Proposed Solution

We are developing a web/mobile-based platform powered by Firebase where users can book nearby house help professionals instantly.

1. User Registration & Login

- Secure authentication using Firebase Authentication
- Email/phone login

- Password encryption handled by Firebase

2. Service Category Selection

- Cleaning
- Plumbing
- Electrician
- Cooking
- Appliance repair

3. Instant Booking

- One-click service request
- Auto assignment to nearest available provider using Firebase Cloud Functions

4. Booking Confirmation

- Unique Booking ID generated automatically
- Confirmation notification via Firebase Cloud Messaging

5. Estimated Arrival Time

- Based on provider location and availability
- Calculated using Google Maps API
- Displayed immediately after booking

6. Booking History

- Past services
- Status tracking
- Re-book option

4. How the System Works

Step-by-Step Workflow

1. **User logs into the application using Firebase Authentication**
2. **Selects a service category**
3. **Clicks on "Book Now"**
4. **Booking request stored in Firebase Firestore**

5. **Firebase Cloud Function detects nearby service providers**
6. **Assigns the closest available provider automatically**
7. **Generates booking ID**
8. **Calculates and displays estimated arrival time**
9. **User receives confirmation notification**
10. **Service provider reaches user location and completes task**
11. **Booking is stored in Firestore history**

5. System Architecture

Components

- **Frontend (User Interface – React.js)**
- **Firebase Authentication (Login & security)**
- **Firestore Database (User, provider, booking data)**
- **Firebase Cloud Functions (Automation logic)**
- **Google Maps API (Location tracking & ETA)**
- **Firebase Cloud Messaging (Notifications)**
- **Firebase Hosting (Deployment)**

6. Tech Stack

Frontend

- **React.js (dynamic UI)**
- **Three.js (3D visualization, optional)**

Backend (Serverless – Firebase)

- **Firebase Authentication**
- **Firebase Firestore Database**
- **Firebase Cloud Functions**
- **Firebase Hosting**

APIs & Services

- **OpenStreetMap (location tracking & ETA)**
- **Firebase Cloud Messaging (notifications)**
- **REST APIs (frontend communication if needed)**

7. Smart Automation Logic

To ensure service within 15 minutes:

- **Location-based provider matching using Firestore data**
- **Priority assignment algorithm based on:**
    - **Distance**
    - **Availability**
    - **Ratings**
- **Automatic fallback if provider rejects request**
- **Real-time ETA calculation using OpenStreetMap**

8.Database

- My SQL Server
  APIs & Services
- OpenStreetMap API (location tracking & ETA)
- Firebase (authentication & notifications)
- REST APIs (communication between frontend & backend)

7. Smart Automation Logic

To ensure service within 15 minutes:

- Use location-based provider matching
- Priority assignment algorithm:
    - Distance
    - Availability
    - Ratings

- Automatic fallback if provider rejects
- Real-time ETA calculation

8. Database Design (Firebase Firestore)

Users Collection

- **userId**
- **name**
- **email**
- **phone**
- **createdAt**

Service Providers Collection

- **providerId**
- **name**
- **serviceType**
- **location**
- **availability**
- **rating**

Bookings Collection

- **bookingId**
- **userId**
- **providerId**
- **serviceType**
- **status**
- **estimatedArrivalTime**
- **timestamp**

9. Steps to Run the Project

Step 1: Install Requirements

- **Node.js**
- **Firebase CLI**
- **Git**
- **Google Antigravity**